Ders 11 Genişletilmiş Notu: Ağaçlar, İkili Ağaçlar ve İkili Arama Ağaçları

Ders 11 Genişletilmiş Notu: Ağaçlar, İkili Ağaçlar ve İkili Arama Ağaçları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ağaç Veri Yapısı ve Kullanım Alanları
- 2. Temel Terimler: Kök, Düğüm, Kenar, İç Düğüm, Yaprak, Derece
- 3. Genel Ağacın Bilgisayarda Saklanması
- 4. Seviye, Yükseklik, Derinlik, Alt Ağaç ve Orman
- 5. İkili Ağaç ve Saklanması
- 6. Tam İkili Ağaç
- 7. Yüksekliği h Olan İkili Ağaçta Düğüm ve Yaprak Sayıları
- 8. Dengeli İkili Ağaç
- 9. İkili Ağaçta Gezinme: Preorder, Postorder, Inorder
- 10. İkili Arama Ağacı
- 11. İkili Arama Ağacına Ekleme
- 12. İkili Arama Ağacında Arama
- 13. Soru-Cevap: Çift Pointer
- Örnek Sorular
- Dersin Toparlaması

1. Ağaç Veri Yapısı ve Kullanım Alanları

Hoca derse, geçen hafta yığın (stack) ve kuyruk (queue) veri yapılarının işlendiğini hatırlatarak başladı. Bu hafta ağaç (tree) veri yapısı anlatıldı. Hoca, ağaçların ayrık matematik (discrete mathematics) dersinde de görüldüğünü, burada ise kodlamasıyla birlikte ele alınacağını söyledi.

Dizi, bağlı liste, yığın ve kuyruk doğrusal (linear) veri yapılarıdır: Elemanlar erişim sırasında arka arkaya bulunur. Ağaç ise doğrusal olmayan (non-linear) bir veri yapısıdır. Ağaç, elemanlara hiyerarşik olarak erişmek gerektiğinde kullanılır. Bilginin bir hiyerarşiyle gösterilmesi ve saklanması gerekiyorsa ağaç yapısı kullanılır.

Hoca üç örnek verdi:

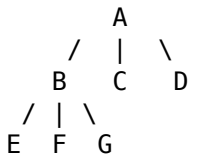
- **Dosya sistemi (file system):** Bilgisayarda bir kök dizin (root directory) vardır. Kök dizine bağlı alt dizinler bulunur. Örneğin “homeworks” altında “data structure”, “discrete mathematics”, “electronics” dizinleri olabilir; “data structure” altında “lecture 1”, “lecture 2” gibi dizinler tutulabilir. “Games” altında Doom, Tetris gibi oyunlar, “documents” altında belgeler olabilir.
- **Aile ağacı:** En yukarıda dede ve anneanne, onların altında çocuklar, onların altında torunlar bulunur.
- **Karar ağacı (decision tree):** Yapay zekâda, makine öğrenmesi altında çok başarılı sonuçlar veren bir öğrenme modelidir; veri madenciliği gibi derslerde görülür. Hocanın örneği hastalık teşhisiydi: “Ateş var mı?” sorusuna evet ya da hayır cevabı verilir. Sonra

"Göz sarı mı?" diye sorulur; evetse (hocanın deyişiyle işler bu kadar kolay olmasa da) sarılık teşhisi konur. Hayırsa öksürük sorulur: az, orta, çok. Bir hasta geldiğinde sorulara verilen cevaplara göre ağaç üzerinde ilerlenir ve teşhise varılır.

Her üç örnekte de bir hiyerarşi vardır. Bundan sonra ağacın özellikleri, bilgilerin ağaçta nerede saklanacağı ve bu bilgilere nasıl erişileceği adım adım incelendi.

2. Temel Terimler: Kök, Düğüm, Kenar, İç Düğüm, Yaprak, Derece

Hoca terminolojiyi ayrık matematikte görülenleri hatırlatmak amacıyla bir örnek ağaç üzerinde verdi. Ağaç, çizim sırasında küçültülüp G eklenerek şu hâle geldi:



- **Düğüm (node):** Ağaçtaki her eleman. Burada A, B, C, D, E, F, G düğümlerdir.
- **Kök (root):** En yukarıdaki eleman. Burada A.
- **Kenar (edge):** Bir düğümü başka bir düğüme bağlayan bağlantı. Kenarın bir değeri olabilir; örneğin karar ağacında "evet" ve "hayır" birer kenardır ve bağlantının bilgisini taşır. Kenar yalnız bağlantıyı da gösterebilir, örneğin "A, B'ye bağlı" demek gibi. Yani kenarın bir değeri olması şart değildir.
- **İç düğüm (internal node):** En az bir çocuğu (alt düğümü) olan düğüm. Bu ağaçta A ve B iç düğümdür.
- **Ebeveyn (parent) ve çocuk (child):** B, C ve D, A'nın çocuklarıdır; A onların ebeveynidir. E, F ve G de B'nin çocuklarıdır.
- **Derece (degree):** Bir düğümün kaç çocuğu olduğu. A'nın ve B'nin derecesi 3'tür; C'nin ve D'nin derecesi 0'dır.
- **Yaprak (leaf):** Hiç çocuğu olmayan düğüm. Burada C, D, E, F, G yapraktır.

Bir öğrenci, sağdaki düğümün (F) çocuklarının da olduğunu, çizimin karıştığını söyledi. Hoca ağacı önce büyük çizdiğini, kâğıtta yer kalsın diye bazı düğümlerin çocuklarını sildiğini açıkladı.

Bu ağaçta bir düğüme bağlanabilecek alt düğüm sayısı belirsizdir: A'ya üç, B'ye üç düğüm bağlı, ama istenildiği kadar alt düğüm bağlanabilir. Bu durum, ağacın bilgisayarda nasıl saklanacağı sorusunu doğurur.

3. Genel Ağacın Bilgisayarda Saklanması

Bir ağaç bilgisayarda iki şekilde gösterilebilir: dizi (array) kullanarak ya da daha önce alışılan şekilde bağlı liste (linked list) kullanarak.

3.1 Dizi ile Saklama

Ağacı diziyle saklamak için bir varsayım yapmak gerekir. Hocanın örneğinde varsayım şuydu: Kök dışındaki her seviyede bir düğüme en fazla 3 çocuk bağlanabilir. Bu varsayımla her düğümün altında üç göz ayrılmış kabul edilir. Olmayan çocuklar için de göz ayrılır ve boş bırakılır. Elemanlar seviye seviye, soldan sağa diziye yazılır: Kök 0. gözdedir, onun üç çocuğu 1, 2, 3. gözlerdedir, sonra bunların çocukları gelir. Hocanın örneğinde bazı gözler boş kaldı ve 13. gözde bir eleman bulunuyordu.

Bu yöntemin iki sakıncası vardır:

- **En fazla çocuk sayısı önceden bilinmelidir.** 3 denirse dördüncü bir çocuk asla bağlanamaz. Dosya sisteminde bunun garantisi yoktur: Köke bağlı 20 dizin olabilir, "documents" altında 5 dizin olabilir. Köke 21. bir dizin bağlanmak istenirse "bağlanamaz" demek gerekir. Bu, statik dizi kullanımında alıştığımız gibi olabilecek en büyük sayıyı önceden düşünmek demektir.

- **Çok sayıda boş göz kalır.** En fazla çocuk sayısı belirlense bile olmayan düğümler için yer tutmak zorunludur. Çünkü bir süre sonra o boş yerlere yeni elemanlar (hocanın örneğinde bir X, bir Z) gelebilir ve ağaç hiyerarşisinin bozulmaması için yerleri hazır olmalıdır.

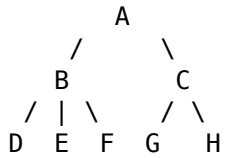
  Ek bilgi (derste anlatılmadı): Her düğümün en fazla k çocuğu olduğu varsayılıp indisler 0'dan başlatılırsa, i. gözdeki düğümün çocukları $ki + 1, ki + 2, \ldots, ki + k$ numaralı gözlerdedir. $k = 3$ için 0. gözün çocukları 1, 2, 3; 1. gözün çocukları 4, 5, 6; 3. gözün çocukları 10, 11, 12 olur. İkili ağaç için derste verilen $2i + 1$, $2i + 2$ formülü bunun $k = 2$ hâlidir.

3.2 Bağlı Liste ile Saklama: Sol Çocuk, Sağ Kardeş

Hocaya göre bütün gözlerin doluluğu konusunda bir garanti yoksa, çok belirgin ve statik bir ağaç yoksa, dizide çok boşluk kalacaksa doğru yaklaşım bağlı liste kullanmaktır. Bağlı listede bir seviyede belirsiz sayıda düğüm olması problemi çözülebilir.

> **Hocanın vurgusu:** Çocuk sayısı belli olmayan bir ağacı saklamak için en uygun gösterim bağlı listedir.

Her düğümde bir değer ve iki bağlantı tutulur: düğümün sol çocuğu (left child) ve aynı seviyede sağındaki kardeşi (right sibling). Hocanın örnek ağacı A'dan H'ye sekiz düğümlüydü:



Bağlı liste gösteriminde bağlantılar şöyledir:

Düğüm	Sol çocuk (left child)	Sağ kardeş (right sibling)
A	B	NULL
B	D	C
C	G	NULL
D	NULL	E
E	NULL	F
F	NULL	NULL
G	NULL	H
H	NULL	NULL

A'nın sol çocuğu B'dir. B'nin sağ kardeşi C'dir, çünkü C, B ile aynı seviyededir. B'nin sol çocuğu D'dir; D'nin sağ kardeşi E, E'nin sağ kardeşi F'dir. C'nin sol çocuğu G'dir. Kardeşi ya da çocuğu olmayan bağlantılar NULL'dır.

Bu gösterimin kolaylığı dinamik olmasıdır. Örneğin E silinmek istenirse E kaldırılır ve D doğrudan F'ye bağlanır. Önemli olan, her seviyede istenen sayıda düğüm ekleyebilmektir; dizideki gibi önceden bir en fazla sayı belirlemek gerekmez.

Hoca bu düğüm için şu yapıyı (structure) verdi. `leftChild` alt seviyedeki en soldaki düğümü, `rightSibling` aynı seviyede sağdaki düğümü gösteren pointer değişkenlerdir:

```
struct node {
    int value;
    struct node *leftChild;
    struct node *rightSibling;
};
```

4. Seviye, Yükseklik, Derinlik, Alt Ağaç ve Orman

4.1 Seviye

Hoca bölüm 2'deki ağacı (A kök; B, C, D birinci seviye; E, F, G ikinci seviye) yeniden çizdi. Kökün bulunduğu seviye 0. seviyedir, altı 1. seviye, onun altı 2. seviyedir. Ağaçta seviyelerin bir

anlamı vardır: Aile ağacında kardeşler aynı seviyede çizilir, babaanne ile dede aynı seviyededir. B, C ve D aynı özellikte oldukları için aynı seviyededir. Dosya sisteminde de “documents”, “homeworks” ve “games” köke bağlıdır ve aynı seviyededir.

4.2 Yükseklik

Bir düğümün yüksekliği (height), o düğümün en uzaktaki yaprağa uzaklığıdır. Yaprak 0 sayılır, bir yukarısı 1, bir yukarısı 2 olur. A düğümünden en uzak yaprağa uzaklık 2 olduğu için A’nın yüksekliği 2’dir. Kökün yüksekliği ağacın yüksekliğidir; bu ağacın yüksekliği 2’dir. D bir yaprak olduğu için yüksekliği 0’dır. B’nin yüksekliği 1’dir.

4.3 Derinlik

Derinlik (depth) kök düğümden uzaklıkla hesaplanır. Kökten (A) en uzaktaki düğümler E, F ve G’dir; hepsi 2 mesafededir, dolayısıyla ağacın derinliği de 2’dir. E’nin derinliği 2, B’nin derinliği 1’dir.

4.4 Alt Ağaç ve Orman

Bir düğüm ve altındaki düğümler bir alt ağaç (subtree) oluşturur. Hoca örneğin B ve çocuklarını bir alt ağaç olarak gösterdi. Tek başına bir yaprak da bir alt ağaçtır. Hangi parçanın alt ağaç olarak alınacağı kullanım amacına bağlıdır.

Birbirinden bağımsız ağaçlar aynı uzayda birlikte bulunursa bir orman (forest) oluştururlar. Hocanın örneği farklı ailelerdi: Her ailenin aile ağacı ayrı, bağımsız bir ağaçtır; bunlar birlikte bir orman meydana getirir.

5. İkili Ağaç ve Saklanması

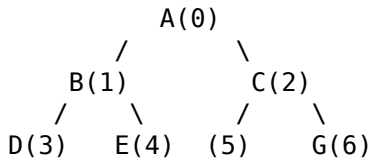
5.1 İkili Ağacın Kullanım Alanları ve Tanımı

Hoca, ağaçların en çok kullanılan şeklinin ikili ağaç (binary tree) olduğunu söyledi. En temel kullanımı ikili arama ağaçlarıdır. Veri tabanlarında da kullanılır. Ayrık matematikte görülen Huffman ağacı (Huffman tree) da ikili ağaçtır. Hocanın belirttiğine göre görüntü için kullanılan JPEG ve video için kullanılan MPEG dosya biçimlerinde, kayıpsız sıkıştırma için kullanılan yöntemlerden biri Huffman kodlamasıdır (Huffman coding). Öncelikli kuyruklarda kullanılan yığın ağacı (heap tree) da ikili ağaç mantığını kullanır.

İkili ağaçta bir köke bağlı bir sol alt ağaç ve bir sağ alt ağaç vardır. Bu, bir düğümün en fazla iki alt düğümü olabileceği demektir. İki çocuk olmak zorunda değildir: Bir çocuk da olabilir, hiç çocuk olmayabilir. En fazla iki çocuğu olabilen ağaçlara ikili ağaç denir.

5.2 Dizi ile Saklama

İkili ağacın bilgisayarda nasıl saklanacağı, ağacın hangi amaçla ve nasıl kullanılacağına bağlıdır. Hoca önce çok boşluk içermeyen bir ağaç aldı:



Elemanlar bir seviye bitince bir sonraki seviyeye geçilerek sırayla saklanır: A, B, C, D, E, boş, G. Gözler 0’dan numaralanırsa:

$$\text{sol çocuk}(i) = 2i + 1, \qquad \text{sağ çocuk}(i) = 2i + 2$$

- 1. düğümün (B) sol çocuğu $2 \cdot 1 + 1 = 3$ (D), sağ çocuğu $2 \cdot 1 + 2 = 4$ (E).
- 2. düğümün (C) sol çocuğu $2 \cdot 2 + 1 = 5$ (boş), sağ çocuğu $2 \cdot 2 + 2 = 6$ (G).

Az boşluk olacağı bilinen durumlarda dizi gösterimi gayet uygundur. Şimdi G'nin sağına bir X düğümü eklendiğini düşünelim. G 6. gözde olduğu için X'in adresi $2 \cdot 6 + 2 = 14$ olmalıdır. Arada 7'den 13'e kadar olan gözler boş kalır. X'in de bir sol çocuğu Z eklenirse onun adresi $2 \cdot 14 + 1 = 29$ olur ve Z 29. göze yerleştirilir. Bu kez arada çok daha fazla boşluk oluşur.

Bu boşluklar zorunludur. Çünkü oralara başka elemanlar gelebilir ve onlara yer bırakmak gerekir. Erişim açısından da her düğümün yeri belli olmalıdır: D 3. gözdeyse çocuklarını bulmak için $2 \cdot 3 + 1 = 7$. ve $2 \cdot 3 + 2 = 8$. gözlere bakılır. Yani i. düğümün çocukları $2i + 1$ ve $2i + 2$'de olmak zorundadır ve buna göre boş yer bırakılmalıdır.

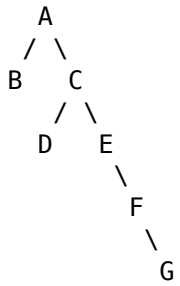
Elemanları arada boşluk olmadan 0, 1, 2, 3, 4, 5 diye arka arkaya duran bir ağaç diziye çok güzel yerleşir. Ama çok boşluklu bir ağaçta dizi verimli bir çözüm değildir. Hangisinin seçileceği uygulamanın ihtiyacına göre karar verilecek bir şeydir.

5.3 Bağlı Liste ile Saklama

Çok boşluk varsa bağlı liste kullanılır. Düğümde yine bir değer bulunur; bu kez sol ve sağ çocukları göstermek için iki pointer vardır:

```
struct node {
    int value;
    struct node *left;
    struct node *right;
};
```

Hocanın örnek ağacında A kök, A'nın çocukları B ve C, C'nin çocukları D ve E, E'nin sağ çocuğu F, F'nin sağ çocuğu G'dir:



Bu ağaç dizide saklansaydı bilgiler 0, 1, 2, 5, 6, 14 ve 30. adreslere yerleşmeliydi (ara adım eklendi: C 2. gözde, çocukları $2 \cdot 2 + 1 = 5$ ve $2 \cdot 2 + 2 = 6$; E'nin sağ çocuğu $2 \cdot 6 + 2 = 14$; F'nin sağ çocuğu $2 \cdot 14 + 2 = 30$). Yani en az 31 gözlü bir diziye ihtiyaç vardır, ama içinde yalnız 7 eleman bulunur. Ağaç büyüdükçe bu böyle devam edebilir.

Bağlı listede ise her düğüm yalnız kendi alt ağaçlarını gösterir. A'nın solu B'yi, sağı C'yi gösterir; B'nin solu ve sağı NULL'dır. C'nin solu D'yi, sağı E'yi gösterir; D'nin iki bağlantısı da NULL'dır. E'nin solu NULL, sağı F'dir; F'nin solu NULL, sağı G'dir; G'nin iki bağlantısı da NULL'dır. Bu şekilde yer kullanımı açısından daha verimli bir gösterim elde edilir. Bir başka avantajı da eleman eklemenin kolaylığıdır; örneğin D'ye bir çocuk eklemek istendiğinde rahatlıkla eklenebilir.

> **Hocanın vurgusu:** Hangi gösterimin kullanılacağına ihtiyaca göre, artı ve eksileri değerlendirilerek karar verilir. İkili ağaçta çok fazla boş eleman olacağı biliniyorsa bağlı liste tercih edilmelidir.

6. Tam İkili Ağaç

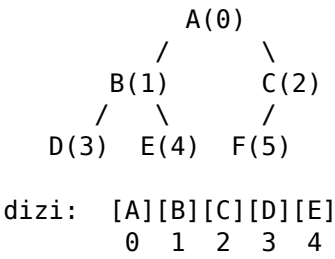
Hoca bu kısımda ikili ağaç terminolojisine geçti. Yüksekliği h olan bir ikili ağacın tam ikili ağaç (complete binary tree) olması için iki şart vardır:

1. 0. seviyeden $h - 1$. seviyeye kadar bütün seviyeler doludur; yani bu seviyelere kadar her düğümün iki çocuğu vardır.
2. Son seviyedeki düğümler sola dayalıdır; aralarında boşluk yoktur.

Hoca bu tanımı birkaç örnekle gösterdi:

- Yüksekliği 2 olan bir ağaçta 0. ve 1. seviyeler doluysa ve son seviyedeki düğümler soldan başlayarak boşluksuz diziliyse ağaç tam ikili ağaçtır.
- 1. seviyede bir düğüm eksikse birinci şart sağlanmaz.
- Son seviyede düğümler arasında boşluk varsa (örneğin bir düğümün çocukları yokken sağındaki düğümün çocukları varsa) ikinci şart sağlanmaz.
- İlk şartı sağlayan ama son seviyesi B'nin çocuklarından başlamayan bir ağaç da tam ikili ağaç değildir. Sola dayalı olmak, son seviyenin mutlaka en soldaki düğümün (B'nin) çocuklarından başlaması demektir.

Tam ikili ağacın önemli bir sonucu vardır. Adreslere bakıldığında düğümler 0, 1, 2, 3, 4, 5 diye arka arkaya gider:



Bu ağaç diziye A, B, C, D, E, F olarak yerleşir ve arada hiç boş göz kalmaz. Bu yüzden tam ikili ağaç için bağlı liste kullanmaya gerek yoktur; dizi yeterlidir.

7. Yüksekliği h Olan İkili Ağaçta Düğüm ve Yaprak Sayıları

Hoca 0, 1, 2 seviyeli, düğümleri 0'dan 5'e numaralanmış bir ikili ağaç üzerinden ilerledi. İki seviye aşağı inildiği için bu ağacın yüksekliği $h = 2$'dir. Yüksekliği h olan bir ikili ağaç için:

Büyüklük	Değer	$h = 2$ için
En çok yaprak sayısı	2^h	4
En çok düğüm sayısı	$2^{h+1} - 1$	7
En az yaprak sayısı	1	1
En az düğüm sayısı	$h + 1$	3

- **En çok yaprak:** Örnek ağaçta eksik olan 6. düğüm de olsaydı ağacın 4 yaprağı olurdu. Yüksekliği 2 olan ağaç için en çok yaprak $2^2 = 4$'tür.
- **En çok düğüm:** Dolu ağaçta düğümler sayılınca 1, 2, ..., 7 tane çıkar. Formül de $2^{2+1} - 1 = 7$ verir.
- **En az yaprak:** Soru "herhangi bir ağaç" için sorulduğunda en az yaprak sayısı 1'dir. Ağaç tek bir kökten oluşuyorsa bu yaprak kökün kendisidir; her düğümün tek çocuğu olan bir zincirde de tek yaprak vardır.
- **En az düğüm:** $h + 1$'dir. Yükseklik 0 ise, yani yalnız kök varsa en az 1 düğüm vardır. Yükseklik 1 ise en az 2 düğüm vardır. Hoca bu açıklamada önce yanlış bir eşleme yaptı, sonra "yükseklik 1 ise en az 2, yükseklik 0 ise 1" diye düzeltti.

Formüller $h = 0$'dan $h = 4$'e kadar python ile sayılarak doğrulandı (örneğin $h = 3$ için en çok 8 yaprak ve 15 düğüm, en az 4 düğüm).

8. Dengeli İkili Ağaç

Hoca dengeli ikili ağacı (balanced binary tree) şöyle tanımladı. Ağacın seviyeleri aşağıdan yukarı doğru derinlik n, $n - 1$, $n - 2$ diye adlandırılır; yani en alttaki seviye ağacın derinliği olan n'dir. Bir ikili ağacın dengeli olması için derinliği $n - 2$'ye kadar olan bütün düğümlerin iki çocuğu olmalıdır. Başka bir kural yoktur.

Yani $n - 2$ seviyesine kadar her düğümün iki çocuğu varsa, aşağıdaki düğümlerin kaç çocuğu olduğuna bakılmaz. Son seviyede istenen yerde bir ya da birkaç düğüm bulunabilir; ağaç yine dengelidir. Tam ikili ağaçtan farkı, son seviyede sola dayalı olma şartının olmamasıdır.

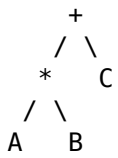
Hocanın ikinci örneğinde $n-2$ seviyesindeki bir düğümün iki çocuğu yoktu. Bu durumda şart sağlanmaz ve ağaç dengesizdir (unbalanced).

> **Ek bilgi (derste anlatılmadı):** "Dengeli ağaç" kaynaklara göre farklı tanımlanır. Örneğin AVL ağacında her düğümde sol ve sağ alt ağaçların yükseklik farkı en çok 1'dir. Bu derste esas alınan tanım, hocanın verdiği "son iki seviye dışında her düğümün iki çocuğu vardır" tanımıdır.

9. İkili Ağaçta Gezinme: Preorder, Postorder, Inorder

Hoca ikili ağaçta gezinmeyi (traversal) anlattı. Ayrık matematikte görülen prefix, infix ve postfix gösterimler ve yığınlar konusunda görülen aritmetik ifade çözümü burada da karşımıza çıkar. $A + B$ gibi bir ifade kökte `+`, solda `A`, sağda `B` olan bir ağaçla gösterilir. Bu ağacın üç farklı gezinme sırası vardır.

Örnek ifade ağacı $(A * B) + C$'dir:



9.1 Preorder

Preorder gezinmede önce düğümün kendisi, sonra sol alt ağacı, sonra sağ alt ağacı ziyaret edilir: kök, sol, sağ. $A + B$ için sonuç `+ A B` olur; bu prefix gösterimdir.

Hoca kodu, ağaçtaki bilgileri preorder sırayla ekrana yazdırmak üzere yazdı. Düğüm değeri kolaylık olsun diye tam sayı (`int`) alındı; karakter de alınabilirdi, o zaman `%c` ile yazdırılırdı. Gezinme, gelen düğüm NULL olana kadar sürer:

```
void preorder(struct node *node)
{
    if (node != NULL) {
        printf("%d ", node->value);
        preorder(node->left);
        preorder(node->right);
    }
}
```

Hoca kodu $(A * B) + C$ ağacı üzerinde adım adım izledi. Beklenen çıktı `+ * A B C`'dir:

1. Kök + NULL değildir; + yazılır. Sol için fonksiyon çağrılır.
2. * NULL değildir; * yazılır. Sol için çağrılır.
3. `A` NULL değildir; `A` yazılır. Solu NULL'dır, döner. Sağı NULL'dır, döner. Yukarı çıkılır.
4. *'ın sağı için çağrılır: `B` yazılır. Solu ve sağı NULL'dır. * bitmiştir, +'ya çıkılır.
5. +'nın sağı için çağrılır: `C` yazılır. Solu ve sağı NULL'dır.

Sonuç: `+ * A B C`. Mantık şudur: Önce kök işlenir (1), sonra sol alt ağaç bitirilir (2), sonra sağ alt ağaç bitirilir (3).

9.2 Postorder

Postorder gezinmede sıra sol, sağ, kök şeklindedir. $A + B$ için sonuç `A B +` olur; bu postfix gösterimdir.

```
void postorder(struct node *node)
{
    if (node != NULL) {
        postorder(node->left);
        postorder(node->right);
        printf("%d ", node->value);
```

```
    }
}
```

Hocanın $(A * B) + C$ üzerindeki izlemesi:

1. + NULL değildir; sol için çağrılır. * NULL değildir; sol için çağrılır. `A` NULL değildir; solu ve sağı NULL'dır. `A` yazılır.
2. *'a çıkılır, sağı için çağrılır. `B`'nin solu ve sağı NULL'dır. `B` yazılır.
3. *'a çıkılır. Solu ve sağı bittiği için * yazılır. Böylece +'nın sol alt ağacı bitmiştir.
4. +'nın sağına geçilir. `C`'nin solu ve sağı NULL'dır. `C` yazılır.
5. +'nın solu ve sağı bittiği için + yazılır.

Sonuç: `A B * C +`. Önce sol bitirilir, sonra sağ bitirilir, en son kök yazılır.

> **Ders sonrası not:** Hoca postorder'ı anlatmaya başlarken (01:26:13) $(A * B) + C$ için "artı çarpı A B C diye yazardık" dedi. Bu, preorder sonucudur. Doğru postorder sonucu `A B * C +`'dır; hoca adım adım izlemede bu sonucu buldu. Hoca izleme sırasında öğrencilerin hatayı fark edip söylemesini beklediğini de belirtti.

9.3 Inorder

Inorder gezinmede sıra sol, kök, sağ şeklindedir. $A + B$ için sonuç alışılan infix ifade `A + B` olur. $(A * B) + C$ için önce sol bitirilir: +'nın solu *, *'ın solu `A`; `A` yazılır. Sonra ortası *, sonra *'ın sağı `B` yazılır; sol alt ağaç `A * B` olarak bitmiştir. Sonra kök +, sonra sağdaki tek eleman `C` yazılır. Sonuç `A * B + C`'dir.

Hoca inorder programını öğrencilerin, preorder ve postorder kodlarına bakarak kendilerinin yazmasını istedi. Çözüm Örnek Sorular bölümündedir.

Üç gezinme kodu ve ifade ağacı `gcc -Wall` ile derlenip çalıştırıldı; çıktılar `+ * A B C`, `A * B + C` ve `A B * C +` oldu.

10. İkili Arama Ağacı

10.1 İkili Aramayı Hatırlama

Hoca, ikili ağacın farklı uygulama alanlarını (işletim sisteminin dosya sistemi, Huffman ağacı, veri tabanları) hatırlattı. Bunlardan biri de sıralı bilgiler üzerinde arama yapmak için kullanılan ikili arama ağacıdır (binary search tree, BST).

Önce dizideki ikili arama (binary search) hatırlatıldı. Sıralı bir dizide orta gözden başlanır. Aranan eleman 3 ise ve ortadaki 9'dan küçükse, dizinin diğer yarısı tamamen elenir. Kalan kısımda baş ve son indislerine göre yeni orta hesaplanır; hocanın örneğinde baş 0, son 1 olduğu için orta $\lfloor 1/2 \rfloor = 0$ oldu ve 3 bulundu. Dizinin sıralı olmasından faydalanarak ikiye böle böle ilerlenir.

10.2 BST Kuralı

İkili arama ağacı bu mantıkla tasarlanmış bir ağaçtır. Bir düğümde x sayısı varsa:

- x'in sol alt ağacındaki bütün sayılar x'ten küçüktür.
- x'in sağ alt ağacındaki bütün sayılar x'ten büyüktür.

Hocanın örneğinde kök 17'dir. 17'nin solundaki bütün değerler 17'den, sağındaki bütün değerler 17'den büyüktür. 17'nin sol çocuğu 11'in solundakiler 11'den küçük, sağındakiler büyüktür. 17'nin sağ çocuğu 19'un solu 19'dan küçük, sağı 19'dan büyüktür. Ağaçta 7 eleman vardır.

10.3 Azalt ve Yönet: 18'i Arama

Bu yapı bir azalt ve yönet (decrease and conquer) yaklaşımıdır. Aranan eleman 18 olsun:

1. 18, 17'den büyüktür. Sol taraf tamamen gözden çıkarılır; bakmaya bile gerek yoktur. Sağdan devam edilir.
2. 17'nin sağında 19 vardır. 18, 19'dan büyük değildir. 19'un sağ tarafına hiç bakmaya gerek yoktur. Soldan devam edilir.

3. 19'un solunda 18 vardır. 18 = 18, aranan eleman bulunmuştur.

   Ders sonrası not: Hoca 7 elemanlı ağaçta elemanın "iki defa sadece bakarak" bulunduğunu söyledi. Kökten iki kez aşağı inilmiştir; karşılaştırma yapılan düğüm sayısı kök dahil 3'tür (17, 19, 18).

10.4 Karmaşıklık

Karmaşıklık ikili aramadakiyle aynıdır: n eleman varsa arama $O(\log n)$ sürer. Arama en fazla ağacın yüksekliği kadar adımda biter.

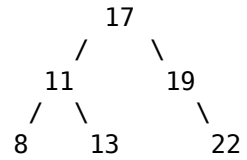
Ancak bu, ağaç dengeliyse geçerlidir. Bir süre eklemeler ve silmeler yapılıp ağaç bozulmuş, bütün elemanlar tek tarafa (örneğin hep sola) dizilmiş olabilir. Bu da bir ikili arama ağacıdır, ama aranan elemanı bulmak için bütün elemanlara sırayla tek tek bakmak gerekir. En kötü durumda (worst case) karmaşıklık $O(n)$ olur.

> **Hocanın vurgusu:** İkili arama ağacının dengesinin bozulmamasına, bozulduysa yeniden düzeltilmesine dikkat edilmelidir.

10.5 BST Nerede Saklanmalı?

Hazır bir ağaç varsa ve hep o kullanılacaksa, dengesi ayarlanıp dizide saklanabilir; hiçbir sıkıntı yoktur. Az yer tutar, erişimi kolay olur. Ama sürekli eleman ekleme, çıkarma, silme oluyorsa, yani bilgi hareketliyse, ağacı bağlı liste üzerinde saklamak bu işlemleri kolaylaştırır. Düğüm yapısı bölüm 5.3'teki tam sayı değerli `struct node`'dur.

Hocanın bağlı liste örneği şu ağaçtı: 17 kökte; 17'nin solunda 11, 11'in solunda 8, sağında 13; 17'nin sağında 19, 19'un solu NULL, sağında 22; 22'nin solu ve sağı NULL.



11. İkili Arama Ağacına Ekleme

11.1 Ekleme Her Zaman Yaprağa Yapılır

Bağlı liste üzerinde ekleme (insert) ve silme (delete) yapılır. Hoca önce eklemeyi anlattı. Ekleme her zaman en aşağıya yapılır. Örneğin 10 geldiğinde 11 ile 8'in arasına eklemeye kalkılırsa yapı bozulur; 11'in artık 10'u, 10'un 8'i göstermesi gibi ek düzenlemeler gerekir. Kolaylık açısından düğümler her zaman en aşağıya eklenir: Karşılaştırarak yaprağa kadar inilir ve yeni düğüm yaprağın soluna ya da sağına bağlanır.

> **Hocanın vurgusu:** Araya eklemeye çalışmıyoruz. Yaprağı bulup yaprağa ekliyoruz, çünkü araya eklemek masraflı bir iştir.

Hocanın bölüm 10.5'teki ağaç üzerinde yaptığı eklemeler:

Eklenen	Yol	Sonuç
10	10 < 17 sola; 10 < 11 sola; 10 > 8	8'in sağına
18	18 > 17 sağa; 18 < 19 sola; 19'un solu NULL	19'un soluna
20	20 > 17 sağa; 20 > 19 sağa; 20 < 22 sola; 22'nin solu NULL	22'nin soluna
6	6 < 17 sola; 6 < 11 sola; 6 < 8 sola; 8'in solu NULL	8'in soluna

> **Ders sonrası not:** 18'i eklerken hoca (01:59:36) "18 17'den büyük sola geldim, 19'dan küçük sağa geldim" dedi. Doğrusu: 18 > 17 olduğu için sağa, 18 < 19 olduğu için sola gidilir ve 18, 19'un soluna eklenir. Dört ekleme C koduyla yapıldı; ağacın inorder çıktısı `6 8 10 11 13 17 18 19 20 22` oldu ve bağlantılar tablodakiyle aynı çıktı.

Araya eklemenin neden ek iş olduğunu hoca 20 üzerinden de açıkladı: 20, 19 ile 22'nin arasına da eklenebilirdi; o zaman 22 çıkarılır, 19 20'yi, 20 de 22'yi gösterir. Ya da 20 doğrudan 17'nin altına konup 19 onun soluna eklenebilir. Bunlar yapılabilir ama hep ekstra iş gerektirir.

11.2 Soru-Cevap: Kök Değiştirilmeli mi?

Soru: Ağaca çok fazla sayı eklendiğini ve hepsinin 17'den büyük olduğunu varsayalım. Sağa çok fazla yükleme yapılıyor. Bir noktada kökü değiştirmek gerekmez mi?

Cevap: Hoca bunun çok doğru bir tespit olduğunu söyledi. Örneğin 11 varken 12, 15, 17, 19 geldikçe başka çare olmadığı için hep sağa eklenir ve denge bozulur. Bu durumda ekleme karmaşıklığı da $O(n)$'e çıkar, arama da $O(n)$ olur. Bu yüzden hareketli ikili arama ağaçlarında belli aralıklarla ağacın durumu kontrol edilip dengesi yeniden sağlanmalıdır.

Hoca bu soru üzerine üç durumu karşılaştırdı:

- **En iyi durum (best case):** Dengesi çok iyi olan bir ağaç. Karmaşıklık $O(\log n)$'dir.
- **Ortalama durum (average case):** Biraz bozulmuş ama çok kötü olmayan bir ağaç. Karmaşıklık yaklaşık $O(\log n)$'dir.
- **En kötü durum (worst case):** Her şeyin bir tarafa gittiği ağaç, örneğin 12, 21, 25, 29 diye hep sağa dizilmiş düğümler. Aranan eleman için en sona kadar gitmek gerekebileceğinden karmaşıklık $O(n)$'dir.

Ağaca denge kazandırmanın yolları vardır. Hoca bunların programda olmadığını, haftalar içinde zaman kalırsa anlatılacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Dengeyi her ekleme ve silmeden sonra kendiliğinden koruyan ağaçlara kendini dengeleyen (self-balancing) arama ağaçları denir; AVL ağacı ve kırmızı-siyah ağaç (red-black tree) bunun bilinen örnekleridir. Bu ağaçlarda dengeyi düzeltmek için döndürme (rotation) işlemleri kullanılır ve arama, ekleme, silme $O(\log n)$'de kalır.

11.3 Ekleme Algoritması

Hoca ekleme algoritmasını adım adım kurdu. Fonksiyona, aramaya kökten başlanacağı için kökün adresi ve eklenecek değer x parametre olarak verilir. İlk iş, `node` tipinde geçici bir düğüm (`tmp`) için yer açmak ve değeri yerleştirmektir. Örneğin $x = 13$ ise `tmp` düğümünde 13 yazar, solu ve sağı NULL'dır. Ayrıca `current` ve `parent` pointer'ları tanımlanır.

Algoritmanın adımları:

1. **Ağaç boş mu?** Kök NULL'ı gösteriyorsa ağaç yoktur. Kök artık `tmp`'yi gösterir.
2. **Boş değilse** kökten başlanıp aşağı inilir: `current = root`. Ayrıca `stop = 0` diye bir durma şartı tutulur; eklenecek yer bulunana kadar dönülür.
3. Döngüde önce nereden gelindiği akılda tutulur: `parent = current`.
4. Değer `current`'ın değerinden küçükse sola gidilir: `current = current->left`. Gidilen yer NULL ise `parent->left = tmp` yapılır ve `stop = 1` olur.
5. Küçük değilse sağa gidilir: `current = current->right`. Gidilen yer NULL ise `parent->right = tmp` yapılır ve `stop = 1` olur.

Kod hâli şöyledir. Hocanın akışındaki adımlar korunmuştur; tek fark, yeni kökün çağırana ulaşması için fonksiyonun kökü geri döndürmesidir (bkz. ders sonrası not):

```
struct node *insert(struct node *root, int x)
{
    struct node *tmp, *current, *parent;
    int stop = 0;

    tmp = (struct node *) malloc(sizeof(struct node));
```

```
    if (tmp == NULL) exit(1);
    tmp->value = x;
    tmp->left = NULL;
    tmp->right = NULL;

    if (root == NULL)            /* ağaç boş: yeni düğüm kök olur */
        return tmp;

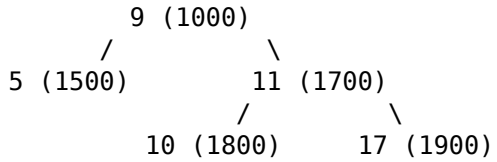
    current = root;
    while (!stop) {
        parent = current;        /* nereden geldiğimizi tut */
        if (x < current->value) {
            current = current->left;
            if (current == NULL) { parent->left = tmp; stop = 1; }
        }
        else {
            current = current->right;
            if (current == NULL) { parent->right = tmp; stop = 1; }
        }
    }
    return root;
}
```

Çağırırken `root = insert(root, 13);` yazılır. Kod `gcc -Wall` ile uyarısız derlendi.

> **Ders sonrası not:** Hoca akışta kök boşsa "root'un artık tmp'yi göstermesini sağlamak" dedi ve kökü fonksiyona parametre olarak aktardı. C'de parametre değerle aktarıldığı için fonksiyon içinde `root = tmp;` yazmak çağırandaki kök değişkenini değiştirmez. Bu yüzden ya yukarıdaki gibi yeni kök geri döndürülmeli, ya kök global bir değişken olmalı, ya da `struct node **root` biçiminde çift pointer kullanılmalıdır.

11.4 Adreslerle İzleme

Hoca algoritmayı adreslerle izledi. Ağaç ve adresler:



Kök 1000 numaralı adrestedir. Yeni düğüm (`tmp`) 800 numaralı adrestedir.

$x = 3$ **eklenirken:**

Adım	`parent`	`current`	Karşılaştırma	Not
Başlangıç		1000 (9)		kök NULL değil
1	1000	1500 (5)	3 < 9, sola	NULL değil, devam
2	1500	NULL	3 < 5, sola	`1500->left = 800`

3, 5'in soluna yerleşti.

$x = 21$ **eklenirken:**

Adım	`parent`	`current`	Karşılaştırma	Not
Başlangıç		1000 (9)		kök NULL değil
1	1000	1700 (11)	21 < 9 değil, sağa	NULL değil, devam
2	1700	1900 (17)	21 < 11 değil, sağa	NULL değil, devam
3	1900	NULL	21 < 17 değil, sağa	`1900->right = 800`

21, 17'nin sağına yerleşti.

Ders sonrası not: 21 izlenirken hoca bir yerde "parent'ımız 1" (doğrusu 1000) ve "1900'ün right'ına 3'ü yazdım" dedi, ardından "21, çok özür dilerim" diyerek düzeltti. 800 adresindeki düğümde bu kez 21 vardır. İki ekleme C koduyla da yapıldı: 9, 5, 11, 10, 17 ağacına 3 ve 21 eklenince 5'in solu 3, 17'nin sağı 21 oldu.

12. İkili Arama Ağacında Arama

Aramada bulunan düğüm geri döndürüleceği için fonksiyonun dönüş tipi `node` tipinde pointer'dır. Parametreleri kök ve aranan tam sayı değerdir. Kökten başlanır. `current` NULL'dan farklı olduğu ve `current`'ın değeri aranan değerden farklı olduğu sürece dönülür. Değer küçükse sola, büyükse sağa gidilir:

```
struct node *search(struct node *root, int value)
{
    struct node *current = root;
    while (current != NULL && current->value != value) {
        if (value < current->value)
            current = current->left;
        else
            current = current->right;
    }
    return current;
}
```

Döngü bitince `return current;` ile düğümün adresi döner. Çağıran taraf aldığı değer NULL ise "aranan sayı bulunamadı" der. NULL'dan farklıysa o adresteki değerle ne yapmak istiyorsa onu yapar.

Hocanın örnek ağacı: 17 kökte (1500 numaralı adres). 17'nin solunda 11 (1000), sağında 22 bulunur. 11'in solunda 6 (800), sağında 13 bulunur; ikisinin de bağlantıları NULL'dır.

13 aranırken: Hoca önce 6'yı aramayı düşündü, sonra bir sola bir sağa gitmek için 13'ü seçti.

1. `current` = 1500, değeri 17. NULL değil, 17 ≠ 13. 13 < 17 olduğu için sola: `current = current->left` = 1000.
2. `current` = 1000, değeri 11. NULL değil, 11 ≠ 13. 13 < 11 değil, sağa: `current = current->right`.
3. `current`'ın değeri 13'tür. Döngü şartı sağlanmaz ve düğümün adresi döndürülür.

9 aranırken:

1. `current` = 1500, değeri 17. 9 < 17, sola: 1000.
2. `current` = 1000, değeri 11. 11 ≠ 9. 9 < 11, sola: 800.
3. `current` = 800, değeri 6. 6 ≠ 9. 9 < 6 değil, sağa: 6'nın sağı NULL'dır.
4. `current` NULL olduğu için döngü biter. NULL döner; aranan sayı bulunamamıştır.

17, 11, 22, 6, 13 değerleriyle kurulan ağaçta C koduyla 13 arandığında düğüm bulundu, 9 arandığında NULL döndü.

13. Soru-Cevap: Çift Pointer

Ders sonunda bir öğrenci konudan biraz farklı bir soru sordu.

Soru: Şimdiye kadarki derslerde bazen dizileri doldururken adresle atayıp çift pointer ile alıp doldurduk, bazen doğrudan tek pointer kullandık. Farkı nedir, hangisini kullanmalıyım?

Cevap: Hoca bu derste hiç çift pointer kullanmadığını, hep tek pointer kullandığını söyledi; bağlı listeye eleman eklerken de normal tek adres gönderilmişti. Çift pointer, pointer'ın pointer'ı demektir. Bu derste yalnız normal değişkenler point edildi. Pointer değişkenleri point etmek gerekirse pointer'ın pointer'ı kullanılır.

Hocanın örneği:

- `a` değişkeni 1250 numaralı adreste durur, değeri 5'tir. Hoca adresle karışmasın diye değeri özellikle 5 seçti.

- `y`, `a`'nın adresini saklar; `y`'nin değeri 1250'dir. `y`'nin kendisi 1800 numaralı adrestedir.
- `x` 1400 numaralı adrestedir ve `y`'nin adresini, yani 1800'ü saklar.
- `b = **x;` dendiğinde: `x`'in içeriğinde 1800 vardır; 1800'ün içeriğinde 1250 vardır; 1250'deki değer getirilir. `b`'ye 5 atanır.

Yani `x`, pointer tipindeki bir değişkenin adresini gösterir; `**x` içeriğin içeriğidir.

```
int a = 5, b;
int *y = &a;     /* y'nin değeri a'nın adresi (1250) */
int **x = &y;    /* x'in değeri y'nin adresi (1800) */
b = **x;         /* 1800 -> 1250 -> 5 */
```

Bu kod derlenip çalıştırıldığında `b` 5 çıktı.

> **Ders sonrası not:** Hoca örnekte "x eşittir y dediğin zaman x'de 1800 saklıyor" dedi. `x`'in `y`'nin adresini tutması için doğru yazım `x = &y;`'dir ve `x`'in tipi `int **` olmalıdır.

Hoca bunun bir programlama dersi konusu olduğunu, nerede gerekebileceğini yalnız örnek olarak gösterdiğini söyledi. Veri yapıları dersinde ilk dersten beri yapılan örneklerde böyle bir ihtiyaç olmadığını, çift pointer kullanmanın öğrencinin kendi tasarımına bağlı olduğunu ekledi.

> **Ek bilgi (derste anlatılmadı):** Çift pointer'ın veri yapılarında en sık görüldüğü yer, bir fonksiyonun çağırandaki baş ya da kök pointer'ını değiştirmesi gereken durumlardır; örneğin boş ağaca ilk düğüm eklenirken `void insert(struct node **root, int x)` yazılır ve içeride `*root = tmp;` denir. Bölüm 11.3'teki kod aynı sorunu yeni kökü geri döndürerek çözer.

Örnek Sorular

Bu derste hazır kare ve video yoktur; sorular derste sözlü sorulan sorulardan alınmıştır.

Soru 1 (derste sözlü, 00:07:03): Dosya sistemi dışında, bilginin hiyerarşik olarak saklanması gereken başka örnekler nelerdir?

> **Çözüm:** Hoca kendisi cevapladı: aile ağacı (dede ve anneanne, çocuklar, torunlar) ve makine öğrenmesindeki karar ağaçları (ör. ateş, göz sarılığı ve öksürük sorularıyla teşhis).

Soru 2 (derste sözlü, 00:56:03): Dizi ile saklanan ikili ağaçta 6. gözdeki G'nin sağ çocuğu X, X'in sol çocuğu Z olursa X ve Z kaçıncı gözlere yerleşir?

> **Çözüm:** X için $2 \cdot 6 + 2 = 14$, Z için $2 \cdot 14 + 1 = 29$. Aradaki gözler, ileride gelebilecek elemanlar için boş bırakılır.

Soru 3 (derste sözlü, 00:56:42): D 3. gözde ise çocuklarını bulmak için hangi gözlere bakılmalıdır?

> **Çözüm:** $2 \cdot 3 + 1 = 7$. ve $2 \cdot 3 + 2 = 8$. gözlere. Hoca önce "altıncı" dedi, sonra yedinci ve sekizinci diye düzeltti.

Soru 4 (derste sözlü, 01:10:16): Yüksekliği h olan bir ikili ağacın en çok kaç yaprağı olabilir?

> **Çözüm:** 2^h. Yüksekliği 2 olan örnek ağaçta en çok $2^2 = 4$ yaprak olur.

Soru 5 (derste sözlü, 01:11:01): Yüksekliği h olan bir ikili ağacın en çok kaç düğümü olabilir?

> **Çözüm:** $2^{h+1} - 1$. $h = 2$ için $2^3 - 1 = 7$; dolu ağaçta düğümler sayılınca da 7 çıkar.

Soru 6 (derste sözlü, 01:11:20): Herhangi bir ağacın en az kaç yaprağı olabilir?

> **Çözüm:** Öğrencilerden "dört" ve "bir" cevapları geldi. Hoca, örnek ağaçta 4 yaprak olsa da herhangi bir ağaç için en az yaprak sayısının 1 olduğunu söyledi; tek düğümlü ağaçta bu yaprak kökün kendisidir.

Soru 7 (derste sözlü, 01:12:30): Yüksekliği h olan herhangi bir ağacın en az kaç düğümü olabilir?

Çözüm: $h + 1$. Yükseklik 0 ise yalnız kök vardır (1 düğüm); yükseklik 1 ise en az 2 düğüm vardır. Her düğümün tek çocuğu olan bir zincirde h yüksekliği için $h + 1$ düğüm yeter.

Soru 8 (derste sözlü, 01:31:54): Inorder (sol, kök, sağ) gezinmenin programını preorder ve postorder kodlarına bakarak yazın.

Çözüm (derste verilmedi): Hoca yalnız $(A*B)+C$ için beklenen çıktıyı (`A * B + C`) verdi. Kod, preorder ile postorder'ın aynı kalıbında yazdırma satırını iki özyinelemeli çağrının arasına koyar. Kod `gcc -Wall` ile uyarısız derlendi; karakter değerli ifade ağacında çıktı `A * B + C`, 17, 11, 19, 8, 13, 22, 10, 18, 20, 6 değerleriyle kurulan BST'de çıktı küçükten büyüğe `6 8 10 11 13 17 18 19 20 22` oldu.

```c
void inorder(struct node *node)
{
    if (node != NULL) {
        inorder(node->left);
        printf("%d ", node->value);
        inorder(node->right);
    }
}
```

Soru 9 (derste sözlü, 02:01:50): Ağaca hep 17'den büyük sayılar eklenirse sağa çok yükleme yapılır; bir noktada kökü değiştirmek gerekmez mi?

Çözüm: Evet. Hep bir tarafa eklendiğinde denge bozulur, ekleme ve arama $O(n)$'e çıkar. Hareketli ikili arama ağaçlarında belli aralıklarla ağacın durumu kontrol edilip dengesi yeniden sağlanmalıdır.

Soru 10 (derste sözlü, 02:24:29): Bazen tek pointer, bazen çift pointer kullanılıyor; farkı nedir, hangisi kullanılmalı?

Çözüm: Çift pointer, pointer'ın pointer'ıdır; bir pointer değişkenin adresini tutar. `a = 5`, `y = &a`, `x = &y` ise `**x` 5'tir. Hoca bu derste böyle bir ihtiyaç olmadığını, kullanımın tasarıma bağlı olduğunu söyledi. Bölüm 11.3'teki ders sonrası nota göre kökün fonksiyon içinde değişmesi gereken eklemede çift pointer kullanmak bir seçenektir.

Dersin Toparlaması

Bu derste doğrusal olmayan ilk veri yapısı olan ağaç ele alındı. Ağaç, dosya sistemi, aile ağacı ve karar ağacı gibi hiyerarşik bilgiyi saklamak ve hiyerarşik erişmek için kullanılır. Kök, düğüm, kenar, iç düğüm, yaprak, derece, seviye, yükseklik, derinlik, alt ağaç ve orman kavramları tanımlandı. Çocuk sayısı belli olmayan genel bir ağacın diziyle saklanmasının en fazla çocuk sayısını önceden bilmeyi gerektirdiği ve çok boş göz bıraktığı görüldü. Bu yüzden sol çocuk ve sağ kardeş bağlantılı bağlı liste gösterimi önerildi. İkili ağaçta dizi gösterimi $2i + 1$ ve $2i + 2$ formülleriyle çalışır. Tam ikili ağaç gibi boşluksuz ağaçlar için dizi uygundur; seyrek ağaçlar için sol ve sağ pointer'lı bağlı liste daha verimlidir. Yüksekliği h olan ikili ağaçta en çok 2^h yaprak ve $2^{h+1} - 1$ düğüm, en az 1 yaprak ve $h + 1$ düğüm bulunur. Preorder, postorder ve inorder gezinmeleri $(A*B)+C$ ifade ağacı üzerinde izlendi ve sırasıyla prefix, postfix ve infix ifadeleri verdi.

Son bölümde ikili arama ağacı anlatıldı. Sol alt ağacın küçük, sağ alt ağacın büyük değerleri tutması sayesinde arama azalt ve yönet mantığıyla, dengeli ağaçta $O(\log n)$'de yapılır. Ağaç tek tarafa yığılırsa arama ve ekleme $O(n)$'e çıkar; bu yüzden hareketli ağaçlarda dengenin belli aralıklarla yeniden sağlanması gerekir. Ekleme her zaman yaprağa yapılır ve `parent`/`current` pointer'larıyla, arama ise `current` NULL olana ya da değer bulunana kadar ilerleyen bir döngüyle gerçekleştirilir. Hoca, ağaca denge kazandırma yöntemlerinin ilerleyen haftalarda zaman kalırsa anlatılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.